T1 - Python Core: input, validation, loops, functions, data structures

Covers the "glue" Python that sits underneath EVERY exam task: reading and validating input, loops, string work, functions, lists/dicts/sets and `random`*. It earns the validation, "loop until q", totals/min/max, and style marks in every past paper.*

1. What the exam asks

Python Core is rarely a task of its own: it is the 1-4 marks hiding inside each task. Past-paper wording that needs it:

Paper	Exact wording	What you need
Practice Task 3	"The user can stop entering data by typing 'q' as the name."	sentinel loop
Practice Task 3	"Prompt the user to enter one or more records of name and height in centimetres (integer)." "Each valid record should be appended..."	validated `int` input
Practice Task 3	"Print the total number of records / the average, minimum, and maximum heights"	count, sum, min, max
2023 Task 1b [1+1+1]	"no duplicate ISBNs are stored"; "year is not in the future AND not earlier than 1450"; "neither the title nor the author fields are empty"	`in`, chained comparison, `.strip()`
2023 Task 1c [3.5+3.5]	"Recommend me a book by [author name]." / "When was [book title] published." "You only need to deal with sentences with those two structures"	string slicing, `.lower()`, `.startswith()`
2022 Task 1b [5]	"Passengers 2 years of age and less ... free (£0). Children between 3 and 12 ... £7 ... seniors aged 66 and over ... £13 ... any other passengers £18." "Display the total cost for the journey"	`if/elif/else` function + running total
2024 Task 1a [3+2]	"Ensure there are no missing values. If any are found, fill them with appropriate values." "Ensure the Total column correctly represents the product of Quantity and Price."	the by-hand logic behind the pandas call
tAPP-3 Problem 5	"If the file does not exist, it should display: File not found. Please try again. and ask for another filename."	`try/except` retry loop
Lab 02 Ex 8	"Display a menu that lets the user add a name, change a name, delete a name, view all names, or end the program."	menu loop in `main()`

NOTE: The 2022/2023 papers used tkinter. That is no longer assessed, but the LOGIC under the buttons (price tiers, year check, parsing a sentence) is still fair game, now as plain `input()`/`print()`.

TIP: The exam also gives marks for style: sensible names, comments, small functions, a `main()`. Core Python done tidily is free marks.

2. Mental model (analogy)

- **Variable** = a labelled box. `total = 0` puts 0 in the box called `total`. `total += 5` takes the number out, adds 5, puts it back.
- `input()` = a paper form handed to the user. Whatever they write comes back as TEXT, even if it looks like a number. `int()` is the clerk who converts the text into a real number (and refuses if it is not one: `ValueError`).
- **Loop** = a conveyor belt. `for` = "do this once per item on the belt". `while` = "keep going until the light turns red (condition False)".
- **Function** = a vending machine. Coins in (parameters), snack out (`return`). Inside, it has its own private room (local variables) nobody outside can see.
- **List** = numbered shelf (0, 1, 2...). **Tuple** = a shelf that is glued shut. **Dict** = a phone book: look up by NAME (key), get the NUMBER (value). **Set** = a bag of unique marbles: no duplicates, no order.

- **Side effect** = lending someone your actual notebook (they can scribble in it). Returning a new list = handing over a photocopy.
- `try/except` = a safety net under the trapeze. If `int("abc")` falls, `except ValueError` catches it instead of crashing the show.

3. Copy-paste templates

3.1 Validated number input (use whenever the spec says "valid")

```
def get_number(prompt, low, high, kind=int):
    """Ask until the user types a number of type kind in [low, high]."""
    while True:
        text = input(prompt)
        try:
            value = kind(text)          # int("42") or float("3.5")
        except ValueError:              # text was not a number
            print("Please type a number, e.g. 42.")
            continue                    # back to the top: ask again
        if low <= value <= high:        # chained comparison
            return value                # only good values leave the loop
        print(f"Must be between {low} and {high}.")

age = get_number("Enter age (0–120): ", 0, 120)
price = get_number("Enter price: ", 0, 1000, float)
print(age, price)
```

3.2 Loop until the user types 'q' (sentinel) and collect records

```
names = []
heights = []
while True:
    name = input("Enter name (or 'q' to quit): ").strip()
    if name.lower() == "q":             # 'Q' and 'q' both quit
        break
    if name == "":                      # reject blank names
        print("Name cannot be empty.")
        continue
    heights.append(get_number("Enter height (cm): ", 50, 250))
    names.append(name)                  # only stored if height was valid
print(f"{len(names)} record(s) entered.")
```

3.3 Count / sum / min / max / average BY HAND (no `min()`, `max()`)

```
heights = [165, 172, 166, 180, 170, 182]
names = ["Amy", "Ben", "Ali", "Yin", "Mohammed", "Gus"]
total = 0
count = 0
highest = heights[0]                    # start from a REAL value, not 0
lowest = heights[0]
tallest_name = names[0]
for name, h in zip(names, heights):     # walk both lists together
    total += h
    count += 1
    if h > highest:
        highest = h
        tallest_name = name             # remember WHO, not just how tall
    if h < lowest:
        lowest = h
if count > 0:                           # avoid ZeroDivisionError
    print(f"Average: {total / count:.2f}")
print(f"Min {lowest}, max {highest} ({tallest_name})")
tall = 0                                # counting items that pass a test
for h in heights:
    if h >= 170:
        tall += 1
print("At least 170cm:", tall)
```

3.4 Build a dict of totals (per product / per category)

```
sales = [("apple", 3), ("pear", 2), ("apple", 5), ("fig", 1), ("pear", 4)]
totals = {}
for product, qty in sales:              # tuple unpacking in the loop
```

```
    totals[product] = totals.get(product, 0) + qty   # 0 if first time
for product in sorted(totals):          # alphabetical by key
    print(f"{product:<8}{totals[product]:>4}")
best = max(totals, key=totals.get)      # key with the biggest value
print("Best seller:", best)
ranked = sorted(totals.items(), key=lambda p: p[1], reverse=True)
print(ranked)                           # [('apple', 8), ('pear', 6), ...]
text = "banana"
letters = {}
for ch in text:                         # same pattern counts occurrences
    letters[ch] = letters.get(ch, 0) + 1
print(letters)                          # {'b': 1, 'a': 3, 'n': 2}
```

NOTE: `lambda p: p[1]` is a tiny nameless function: given a `(key, value)` pair `p`, it returns the value, so `sorted` ranks by value.

3.5 Menu loop in `main()` (Lab 02 Ex 8 style, add/change/delete/view/quit)

```
def main():
    names = []
    while True:
        print("\n1) Add  2) Change  3) Delete  4) View  5) Quit")
        choice = input("Choose 1-5: ").strip()   # str: no int() crash
        if choice == "1":
            names.append(input("Name to add: "))
        elif choice == "2":
            old = input("Name to change: ")
            if old in names:
                names[names.index(old)] = input("New name: ")
            else:
                print("Name not in list.")
        elif choice == "3":
            gone = input("Name to delete: ")
            if gone in names:
                names.remove(gone)
            else:
                print("Name not in list.")
        elif choice == "4":
            for i, name in enumerate(names, start=1):
                print(f"{i}. {name}")
            if not names:
                print("(list is empty)")
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid option, try again.")

if __name__ == "__main__":
    main()
```

3.6 Tiered decision as a function (2022 bus fare) + running total

```
def ticket_price(age):
    """Return the fare in pounds for one passenger."""
    if age <= 2:
        return 0
    elif age <= 12:                     # 3..12 (age <= 2 already handled)
        return 7
    elif age >= 66:
        return 13
    else:
        return 18

ages = [2, 78, 68, 45, 2]
prices = [ticket_price(a) for a in ages]     # list comprehension
print(prices, "total £", sum(prices))        # [0, 13, 13, 18, 0] total £ 44
```

3.7 Retry until a file opens (tAPP-3 Problem 5, fixed)

```
def read_file():
    while True:
        filename = input("Enter filename: ")
        try:
            with open(filename, "r") as file:    # closes the file for you
                return file.read()
        except FileNotFoundError:
            print("File not found. Please try again.")
# text = read_file()      # (needs a real file, so not run here)
```

3.8 Strings: methods and slicing (Lab 01 idioms)

```
s = "  The Signalman  "
t = s.strip()                  # 'The Signalman' (also lstrip / rstrip)
print(t.lower(), t.upper(), t.title())   # the signalman THE SIGNALMAN ...
print(len(t), t[0], t[-1])               # 13 T n   (-1 = last character)
print(t[4:], t[:3], t[4:8], t[::-1])     # Signalman | The | Sign | namlangiS
print(t.replace("The ", ""), t.find("Sig"), "man" in t)   # Signalman 4 True
words = "red green blue".split()         # ['red', 'green', 'blue']
print(" - ".join(words), t.startswith("The"), "42".isdigit())
year = 2023
print("Year: " + str(year))   # + needs str; an f-string is easier
print(f"{3.14159:.2f} | {42:>6} | {'ab':<5}| {1234567:,}")
# 3.14 |     42 | ab   | 1,234,567
```

Slice rule: `s[start:end]` includes `start`, STOPS BEFORE `end`. Lab 01 length/slicing: to show characters start..end inclusive use `s[start:end + 1]`.

3.9 Lab 01 patterns: pig latin, vowels, parse a sentence (2023 Task 1c)

```
word = "banana"
vowels = "aeiou"
if word[0] in vowels:                    # vowel: add "way"
    print(word + "way")
else:                                    # consonant: move 1st letter
    print(word[1:] + word[0] + "ay")     # ananabay

question = "Recommend me a book by Arthur Conan Doyle"
prefix = "recommend me a book by "
if question.lower().startswith(prefix):
    author = question[len(prefix):].strip()     # everything after prefix
    print("Author:", author)                     # Author: Arthur Conan Doyle

question = "When was The Signalman published?"
low = question.lower()
if low.startswith("when was ") and low.endswith(" published?"):
    title = question[len("when was "):-len(" published?")]   # cut both ends
    print("Title:", title)                       # Title: The Signalman
```

3.10 Lists, tuples, dicts, sets, comprehensions (Lab 02 Data Structures)

```
nums = [5, 3, 8, 3]
nums.append(1); nums.insert(0, 9)        # [9, 5, 3, 8, 3, 1]
nums.remove(3); last = nums.pop()        # remove first 3; pop() -> 1
print(nums, nums.index(8), 8 in nums)    # [9, 5, 8, 3] 2 True
print(sorted(nums), sorted(nums, reverse=True), nums)   # nums unchanged
nums.sort()                              # in-place, returns None!
squares = [n * n for n in nums if n > 3]         # [25, 64, 81]
countries = ("India", "Japan", "USA")            # tuple: cannot change
print(countries.index("Japan"), countries[-1])   # 1 USA
person = {"name": "Som", "age": 21}      # dict
person["shoe"] = 9                       # add / change
print(person.get("height", "unknown"))   # no KeyError if key missing
for key, value in person.items():        # loop over pairs
    print(key, value)
person.pop("shoe")                       # delete a key
people = [{"name": "Som", "age": 21}, {"name": "Ash", "age": 26}]
people.sort(key=lambda p: p["age"], reverse=True)    # sort dicts by field
print([p["name"] for p in people])       # ['Ash', 'Som']
seen = set()                             # duplicate check (2023 ISBN rule)
for isbn in ["111", "222", "111"]:
    if isbn in seen:
        print("Duplicate:", isbn)
    seen.add(isbn)
grid = [[2, 5, 8], [3, 7, 4]]
print(grid[1][2], len(grid), len(grid[0]))       # 4 2 3  (row, then column)
```

3.11 `random` (Lab 02 Random exercises)

```
import random
random.seed(1)                          # for testing only: delete in exam
fruits = ["apple", "banana", "mango"]
print(random.choice(fruits))            # one random item from a list
secret = random.randint(1, 5)           # 1 and 5 BOTH possible
coin = random.choice(["h", "t"])
print(secret in range(1, 6), coin in ("h", "t"))     # True True
random.shuffle(fruits)                  # in-place shuffle
print(random.random() < 1)           # a float from 0 up to (not incl.) 1
print(random.randrange(0, 101, 10) % 10 == 0)   # step of 10: True
```

3.12 Functions: defaults, tuple return, scope, side effects (tAPP-2 P6)

```
def box(width, height, fill="*"):       # fill has a DEFAULT value
    return (fill * width + "\n") * height

def min_max(values):
    return min(values), max(values)     # returns a tuple of two values

low, high = min_max([4, 9, 2])          # tuple unpacking
print(box(3, 2), end="")             # two rows of ***
print(low, high)                        # 2 9

scores = [60, 70]
def add_score_a(score):                 # SIDE EFFECT: changes the real list
    scores.append(score)
def add_score_b(old_scores, score):     # no side effect: returns a new list
    return old_scores + [score]
add_score_a(80)
print(scores)                           # [60, 70, 80]
new = add_score_b(scores, 90)
print(scores, new)                      # [60, 70, 80] [60, 70, 80, 90]
```

4. Line-by-line walkthrough

One small program that uses all the exam patterns together: read sales until `q`, validate each quantity, build a dict of totals, print a report. (Assumes `get_number` from 3.1 is defined above it.)

```
def read_sales():                       # one job: collect the data
    totals = {}                         # empty phone book: product -> qty
    while True:                         # belt runs until we break
        product = input("Product (or 'q'): ").strip().lower()
        if product == "q":              # sentinel: user wants out
            break                       # leave the while loop
        if product == "":               # blank is not a product
            print("Product cannot be empty.")
            continue                    # skip the rest, ask again
        qty = get_number("Quantity (1-100): ", 1, 100)  # safe int
        totals[product] = totals.get(product, 0) + qty  # add to running
    return totals                       # hand the dict to the caller

def print_report(totals):               # one job: show the data
    if not totals:                      # empty dict is "falsy"
        print("No sales entered.")
        return                          # leave the function early
    grand_total = 0
    for product in sorted(totals):      # alphabetical order of keys
        qty = totals[product]           # look up the value by key
        grand_total += qty              # running total
        print(f"{product:<10}{qty:>5}") # <10 = left, 10 wide; >5 = right
    print(f"{'TOTAL':<10}{grand_total:>5}")

def main():
    totals = read_sales()               # call 1: gather
    print_report(totals)                # call 2: report

if __name__ == "__main__":              # True only when run directly
    main()                              # (False if file is imported)
```

Typing `Apple`, `3`, (blank), `apple`, `abc`, `4`, `fig`, `0`, `7`, `q` gives (apple 3 + 4 = 7):

```
Please type a number, e.g. 42.      <- for "abc"
Must be between 1 and 100.        <- for "0"
apple         7
fig           7
TOTAL        14
```

Why it is split into functions: `read_sales` never prints a report and `print_report` never reads input. Each can be tested alone, and no function changes a global variable (no side effects). That is exactly the style the unit rewards.

5. Traps & debugging

Mistake	Error / symptom	Fix
`age = input()` then `age > 18`	`TypeError: '>' not supported between instances of 'str' and 'int'`	`age = int(input(...))`
`"Age: " + 21`	`TypeError: can only concatenate str (not "int") to str`	`f"Age: {age}"`
User types `abc` into `int(input())`	`ValueError: invalid literal for int() with base 10: 'abc'`	wrap in `try/except ValueError` inside a `while True` (3.1)
`range(1, 10)` expected to include 10	last number printed is 9	`range(1, 11)`; count down: `range(20, 0, -1)`
`s[2:5]` expected to include index 5	one character short	end is exclusive: `s[2:6]`

Mistake	Error / symptom	Fix
`if x == "a" or "b":`	always True	`if x == "a" or x == "b":` or `x in ("a", "b")`
`if x = 5:`	`SyntaxError`	`==` compares, `=` assigns
`names[5]` on a 3-item list	`IndexError: list index out of range`	check `len()` or use `in`
`d["missing"]`	`KeyError: 'missing'`	`d.get("missing", 0)` or `if key in d`
`x = nums.sort()` then `print(x)`	prints `None`	`.sort()` changes in place; use `sorted(nums)` to get a list
`max(values)` on `[]`	`ValueError: max() ... is empty` (wording varies by Python version)	`if values:` first
`total / count` with `count == 0`	`ZeroDivisionError`	`if count > 0`
Setting `highest = 0` for min/max	wrong answer when all values are negative / `lowest` stays 0	start from the first item
`x += 1` on a global inside a function	`UnboundLocalError: ... local variable 'x' ...` (message wording varies by version)	pass it in, `return` it out
`def f(items=[])`	list is shared between calls	`def f(items=None)` then `if items is None: items = []`
Forgot `f` before the quote: `input("Enter (0–{n}): ")`	prints `{n}` literally	`f"..."` (this bug is in Lab 01 `length_and_slicing.py`)
`while` condition never changes	infinite loop; stop it with Ctrl+C	update the variable inside the loop
Tab/space mix, wrong indent	`IndentationError`	4 spaces everywhere
`for x in lst: lst.remove(x)`	skips items	loop over a copy: `for x in lst[:]`

Quirks spotted in the course's own solutions (do not copy these blindly):

- Lab 02 `19.py`: `if user_idx <= len(countries)` lets `len(countries)` through, then `IndexError`. Use `0 <= user_idx < len(countries)`.
- Lab 02 `30.py`: `print("Name not in Data")` sits outside the `if`, so it ALWAYS prints. Use `for ... else` or a `found` flag.
- Lab 02 `05.py`: `count` starts at 0 and is only increased on a wrong guess, so a correct first guess says "0 attempts". Start at 1.
- Lab 02 `20.py`: `sorted(arr)` on an `array` returns a plain `list` (fine to print, but no longer an `array`).
- Lab 01 `Upper_Lower_Name.py`: the brief says "join ... without a space"; the solution adds one. Always re-read the spec.
- Lab 02 `12.py`: the "clue" prints the whole list plus the answer's position, which gives the answer away.

6. Anticipated exam questions

Q1 [3 marks] Ask for an age. Re-ask until it is a whole number from 0 to 120, then return it. Answer: template 3.1. `age = get_number("Enter age (0–120): ", 0, 120)`. Marks: `try/except ValueError` (1), range check (1), loop re-asks (1).

Q2 [4 marks] Read ages one at a time until the user types `q`. Store valid ages in a list; ignore (with a message) anything that is not a number.

```
ages = []
while True:
    text = input("Age (or 'q'): ").strip()
    if text.lower() == "q":
        break
```

```
    try:
        ages.append(int(text))
    except ValueError:
        print("Not a number, ignored.")
print(ages)
```

Q3 [4 marks] Write `ticket_price(age)` using the 2022 rules and a function `group_cost(ages)` that returns the total.

```
def group_cost(ages):
    total = 0
    for age in ages:
        total += ticket_price(age)       # ticket_price from template 3.6
    return total
print(group_cost([2, 78, 68, 45, 2]))    # 44
```

Q4 [4 marks] Given `rows = [("Amy", 165), ("Ben", 172), ("Amy", 170)]`, build a dict of total height per name and print the name with the largest total. Answer: template 3.4: `totals[name] = totals.get(name, 0) + h`, then `max(totals, key=totals.get)`.

Q5 [3 marks] Without using `min()` / `max()`, find the smallest and largest of a list and the average to 2 d.p. Answer: template 3.3 (start `lowest` / `highest` from `values[0]`, guard `count > 0`, print with `:.2f`).

Q6 [3 marks] The user types `Recommend me a book by Katherine Rundell`. Extract the author and print it in lower case. Answer: template 3.9: `question[len(prefix):].strip().lower()` after checking `question.lower().startswith(prefix)`.

Q7 [2 marks] What does this print, and why?

```
def add(items, x):
    items.append(x)
data = [1, 2]
add(data, 3)
print(data)
```

Answer: `[1, 2, 3]`. The function changes the caller's list (a side effect) because the parameter points at the same list. Returning `items + [x]` instead would leave `data` unchanged.

Q8 [4 marks] Computer picks 1-5 (`random.randint`), user gets two guesses, with "too high/too low" after the first.

```
import random
secret = random.randint(1, 5)
guess = get_number("Guess 1–5: ", 1, 5)
if guess == secret:
    print("Well done")
else:
    print("Too high" if guess > secret else "Too low")
    second = get_number("Guess again: ", 1, 5)
    print("Correct" if second == secret else "You lose")
```

Q9 [2 marks] Find and fix the error: `for i in range(len(names)): print(names[i+1])`. Answer: `IndexError` on the last pass (`i + 1` is past the end). Use `print(names[i])` or `for name in names:`.

Q10 [3 marks] Reject a duplicate ISBN and an empty title. Answer: `if isbn in seen:` print message and `continue`; `if title.strip() == "":` message and `continue`; else `seen.add(isbn)` (template 3.10).

7. Quick-reference box

```
int(input("..."))          text -> whole number (ValueError if not)
float(x)   str(x)          conversions;  f"{x:.2f}"  2 decimals
VALIDATE:  while True: try: v=int(input()) except ValueError: continue
           if low <= v <= high: break          (return v inside a function)
SENTINEL:  while True: x=input(); if x.lower()=="q": break
range(a,b)  a..b-1;  range(20,0,-1) counts down;  range(n) = 0..n-1
s[a:b]  b excluded; s[-1] last; s[::-1] reversed; .strip() .lower() .split()
startswith / endswith / find / replace / "x" in s / .join(list)
total=0; for v in L: total+=v     count/sum;  max: start from L[0]
```

```
d[k] = d.get(k, 0) + 1            count or total per key
sorted(d.items(), key=lambda p: p[1], reverse=True)    sort dict by value
L.append / .remove(v) / .pop() / .sort() (None!) / sorted(L) (new list)
for i, x in enumerate(L, 1)    for a, b in zip(A, B)   [x*2 for x in L if x>0]
random.randint(1,5)  random.choice(L)  (import random)
def f(a, b=0): return a+b      no return -> None;  return x, y = tuple
side effect: list.append inside a function changes the caller's list
if __name__ == "__main__":  main()
```